

1 Missionszentrale – Schülerhandbuch

1.1 1. Was ist neu?

Die Weltraum-Kolonie besitzt jetzt eine **Missionszentrale**. Dort stehen zusätzliche Aufgaben, die während des Spiels erledigt werden können. Missionen sind keine Pflicht-Siegbedingungen. Deshalb kann man auch im freien Spiel Missionen erledigen und Belohnungen sammeln.

Eine Mission besteht immer aus drei Teilen:

Teil	Bedeutung	Beispiel
Bedingung	Was muss passieren?	Ein Gebäude bauen
Fortschritt	Wie viel wurde schon geschafft?	4/10 Gebäude
Belohnung	Was bekommt die Kolonie?	+100 Gold

Die Missionen befinden sich in der Datei `missionen.py`. Der wichtigste Vorteil dieser Struktur ist, dass neue Missionen später einfach als weitere Dictionaries in die Liste `MISSIONEN` eingetragen werden können.

1.2 2. Die Dateien des Systems

Datei	Aufgabe
<code>missionen.py</code>	Missionsdaten, Prüfung, Fortschritt und Missionsmenü
<code>main.py</code>	Liefert aktuelle Spielwerte und zahlt Belohnungen aus
<code>spielstand.py</code>	Speichert und lädt Missionsfortschritt als JSON
<code>test_stunde12.py</code>	Prüft Menü, Missionserfüllung und Speicherung
<code>README.md</code>	Allgemeine Spielbeschreibung und Bedienung

Das Missionsmodul kennt die komplette Spielschleife nicht. Es bekommt von `main.py` nur ein Dictionary mit Zahlen. Dadurch bleibt der Code übersichtlich und die Schülerinnen und Schüler können die Missionen bearbeiten, ohne die gesamte Spielmechanik verstehen zu müssen.

1.3 3. Eine Mission lesen

Ein Eintrag aus der Liste sieht zum Beispiel so aus:

```
{
  "id": "meisterbauer",
  "titel": "Meisterbauer",
  "beschreibung": "Baue insgesamt zehn Gebäude.",
  "bedingungen": [("gebaeude_gesamt", 10)],
  "belohnung": {"gold": 100},
},
```

Die einzelnen Schlüssel haben eine klare Aufgabe:

Schlüssel	Erklärung
id	Eindeutiger Computername. Keine zwei Missionen dürfen dieselbe ID besitzen.
titel	Lesbarer Name, der im Spielmenü erscheint.
beschreibung	Erklärt der Spielperson das Ziel.
bedingungen	Liste aus einem Zählernamen und einem Zielwert.
belohnung	Dictionary aus Ressourcenname und Belohnungsmenge.

Die eckigen Klammern in [("gebaeude_gesamt", 10)] zeigen, dass bedingungen eine **Liste** ist. Dadurch können später auch mehrere Bedingungen verwendet werden.

Beispiel mit zwei Bedingungen:

```
"bedingungen": [
    ("gebaeude_typ_2", 1),
    ("bevoelkerung", 20),
],
```

Diese Mission wäre erst erledigt, wenn sowohl eine Farm gebaut wurde als auch mindestens 20 Bewohner vorhanden sind. Die Bedingungen werden mit **UND** verbunden.

1.4 4. Die vorhandenen Missionen

Die aktuelle Version enthält 15 Missionen.

Mission	Ziel	Belohnung
Erste Kolonie	Erstes Gebäude bauen	50 Gold
Stromversorgung	Einen Reaktor bauen	30 Holz
Grüner Daumen	Eine Farm bauen	30 Stein
Wohnraum schaffen	Drei Wohnhäuser bauen	75 Gold
Forschungslabor	Erste Technologie erforschen	25 Forschung
Meisterbauer	Zehn Gebäude bauen	100 Gold
Handelsabkommen	Drei Handelsaktionen durchführen	75 Gold
Terraformer	Fünf Kacheln umwandeln	50 Nahrung
Stahlzeit	Zehn Stahl besitzen	100 Gold
Roboterhilfe	Fünf Roboter besitzen	25 Kohle
Bevölkerungsboom	25 Bewohner erreichen	50 Energie
Überlebender	20 Wirtschaftsticks erreichen	75 Nahrung

Mission	Ziel	Belohnung
Zentrum der Kolonie	Koloniezentrum bauen	75 Forschung
Vollausbau	20 Gebäude bauen	200 Gold
Missionsmeister	Zehn andere Missionen abschließen	100 Forschung und 100 Gold

1.5 5. Wie der Fortschritt gespeichert wird

Das Modul besitzt zwei wichtige Variablen:

```
_erledigt = set()
_zaehler = {}
```

`_erledigt` ist ein **Set**. In einem Set darf jeder Wert nur einmal vorkommen. Das ist ideal, weil eine erledigte Mission nicht zweimal belohnt werden darf.

```
_erledigt.add("erste_kolonie")
```

Mit dieser Zeile wird eine Mission als erledigt markiert.

`_zaehler` ist ein **Dictionary**. Es speichert Zahlen unter Namen:

```
_zaehler["gebaeude_gesamt"] = 7
_zaehler["terraformierungen"] = 3
```

Der Missionsfortschritt bleibt auch nach dem Start einer neuen Kolonie erhalten. So kann eine Person langfristig Missionen sammeln, ohne eine Belohnung durch ständiges Neustarten mehrfach zu erhalten.

1.6 6. Wie eine Mission geprüft wird

Die Funktion `pruefen(kontext)` erhält die aktuellen Werte:

```
kontext = {
    "gebaeude_gesamt": 7,
    "bevoelkerung": 18,
    "forschungen": 2,
    "handelsaktionen": 1,
}
```

Danach wird jede Mission mit einer `for`-Schleife besucht:

```
for mission in MISSIONEN:
    if mission["id"] in _erledigt:
        continue

    bedingungen_erfuellt = all(
        _wert(kontext, schluessel) >= ziel
```

```

        for schluessel, ziel in mission["bedingungen"]
    )

```

Hier sind mehrere wichtige Python-Grundlagen enthalten:

Python-Baustein	Bedeutung im Beispiel
for	Geht alle Missionen nacheinander durch
if	Prüft eine Entscheidung
continue	Überspringt bereits erledigte Missionen
Dictionary-Zugriff	mission["id"] liest einen Wert aus
all(...)	Prüft, ob alle Bedingungen wahr sind
>=	Vergleicht den aktuellen Wert mit dem Ziel

Wenn alle Bedingungen erfüllt sind, wird die ID in das Set eingetragen:

```

if bedingungen_erfuellt:
    _erledigt.add(mission["id"])
    neue_missionen.append(mission["id"])

```

Die Funktion gibt eine Liste der neuen Abschlüsse zurück. Dadurch weiß `main.py`, für welche Missionen jetzt eine Belohnung ausgezahlt werden muss.

1.7 7. Wie Belohnungen ausgezahlt werden

Das Missionsmodul entscheidet nicht selbst, wie Ressourcen verändert werden. Es liefert nur die Belohnung. `main.py` übernimmt die Auszahlung:

```

neue = missionen.pruefen(missionen_kontext())

for mission_id in neue:
    belohnung = missionen.belohnung(mission_id)
    for ressourcen_name, menge in belohnung.items():
        ressourcen_dict[ressourcen_name] = (
            ressourcen_dict.get(ressourcen_name, 0) + menge
        )

```

Diese Trennung ist ein wichtiges Programmierprinzip:

Jede Funktion sollte möglichst eine klar erkennbare Aufgabe haben.

`missionen.py` prüft Missionen. `main.py` verändert den echten Spielzustand. So ist es leichter, Fehler zu finden.

1.8 8. Wann das Spiel Missionen prüft

Die Prüfung wird an mehreren Stellen aufgerufen:

Ereignis	Warum wird geprüft?
Erfolgreiches Bauen	Gebäude-Missionen können sofort fertig werden.
Erfolgreiches Terraforming	Die Terraformer-Mission erhält Fortschritt.
Wirtschaftstick	Ressourcen, Forschung, Bevölkerung und Handelsaktionen können sich verändern.
Öffnen des Missionsmenüs	Der angezeigte Fortschritt ist aktuell.

Ein neuer Auftrag wird im Spiel als Meldung angezeigt, zum Beispiel:

Mission erledigt: Stromversorgung (+30 Holz)

1.9 9. Das Missionsmenü

Die Missionszentrale wird mit M geöffnet. Im Pausenmenü kann sie zusätzlich mit I geöffnet werden. Esc, M, Enter oder die Leertaste schließen die Übersicht wieder.

Jede Mission zeigt:

1. ein offenes oder erledigtes Symbol,
2. den Titel,
3. die Beschreibung,
4. die Belohnung,
5. den aktuellen Fortschritt.

Beispiel:

○ Meisterbauer
Baue insgesamt zehn Gebäude.
Belohnung: +100 Gold | 4/10

Eine erledigte Mission wird grün dargestellt und zeigt ERLEDIGT an.

1.10 10. Eine eigene Mission ergänzen

Eine neue Mission wird innerhalb der Liste MISSIONEN ergänzt. Ein mögliches Beispiel ist eine Energiemission:

```
{
  "id": "energieprofi",
  "titel": "Energieprofi",
  "beschreibung": "Halte mindestens 80 Energie bereit.",
  "bedingungen": [{"energie", 80}],
  "belohnung": {"gold": 60, "holz": 20},
},
```

Dabei muss auf fünf Dinge geachtet werden:

Prüffrage	Beispiel
Ist die ID eindeutig?	energieprofi
Ist der Titel verständlich?	Energieprofi
Ist die Bedingung als Liste geschrieben?	[("energie", 80)]
Existiert der Zähler im Kontext?	main.py muss energie liefern.
Sind die Ressourcen gültig?	gold und holz existieren bereits.

Wenn ein neuer Zähler verwendet werden soll, muss er in `missionen_kontext()` in `main.py` ergänzt werden.

1.11 11. Mehrere Bedingungen

Eine anspruchsvollere Mission kann mehrere Ziele verbinden:

```
{
    "id": "gruendung",
    "titel": "Gründung einer Stadt",
    "beschreibung": "Baue fünf Gebäude und erreiche 20 Bewohner.",
    "bedingungen": [
        ("gebaeude_gesamt", 5),
        ("bevoelkerung", 20),
    ],
    "belohnung": {"gold": 150},
},
```

Die Funktion `all(...)` sorgt dafür, dass beide Bedingungen erfüllt sein müssen. Wird nur eine Bedingung erfüllt, bleibt die Mission offen.

1.12 12. Häufige Fehler

1.12.1 Die Mission wird immer wieder belohnt

Dann wird die Mission wahrscheinlich nicht in `_erledigt` eingetragen oder der Zustand wird bei jedem Tick zurückgesetzt. Die Zeile

```
_erledigt.add(mission["id"])
```

muss nur beim ersten erfolgreichen Abschluss ausgeführt werden.

1.12.2 Fortschritt bleibt bei 0/10

Dann stimmt der Schlüssel in der Mission wahrscheinlich nicht mit dem Schlüssel im Kontext überein. `gebaeude_gesamt` und `gebäude_gesamt` wären zum Beispiel zwei verschiedene Namen. Python unterscheidet außerdem Groß- und Kleinschreibung.

1.12.3 Die Belohnung erscheint nicht

Dann liefert `pruefen()` zwar eine Mission zurück, aber `main.py` zahlt die Belohnung nicht aus. Prüfe zunächst, ob `missionen_pruefen()` nach der Aktion aufgerufen wird.

1.12.4 Das Spiel stürzt beim Laden ab

Der Spielstand muss auch für ältere Versionen funktionieren. Deshalb werden Missionsdaten nur importiert, wenn das Feld `missionen` im JSON vorhanden ist. Alte Spielstände bleiben dadurch grundsätzlich lesbar.

1.13 13. Eigene Erweiterungsaufgaben

Aufgabe	Lernziel
Ergänze eine Mission für 100 Holz.	Dictionaries und Bedingungen
Ergänze eine Mission mit zwei Bedingungen.	Listen und <code>all(...)</code>
Ändere eine Belohnung.	Dictionary-Werte verändern
Baue eine Mission für eine bestimmte Gebäudesorte.	Schlüssel wie <code>gebaeude_typ_2</code> verwenden
Ergänze einen neuen Zähler.	Daten von <code>main.py</code> an ein Modul übergeben
Füge eine dritte Spalte im Missionsmenü hinzu.	Schleifen und Bildschirmkoordinaten
Speichere zusätzlich einen Missionsrang.	JSON und Zustandsverwaltung

1.14 14. Testen

Nach Änderungen sollte das Projekt aus dem Projektordner heraus getestet werden:

```
python3 -m py_compile main.py spiel_menuue.py spielstand.py achievements.py
missionen.py
python3 test_stunde11.py
python3 test_stunde12.py
```

Bei einem erfolgreichen Lauf erscheinen:

```
STUNDE11_TESTS_OK
STUNDE12_TESTS_OK
```

Der neue Test prüft unter anderem, ob das Missionsmenü geöffnet werden kann, ob die erste Mission eine Belohnung auszahlt und ob die erledigte Mission in der JSON-Struktur des Spielstands landet.

1.15 15. Gute nächste Erweiterungen

Im späteren Kurs könnten Schülerinnen und Schüler Missionen mit Zeitlimits, zufälligen Ereignissen oder verschiedenen Schwierigkeitsstufen ergänzen. Eine weitere sinnvolle Erweiterung wäre ein „Tagesauftrag“, der bei jedem neuen Spiel wechselt, während die dauerhaft gesammelten Missionen weiterhin im Profil erhalten bleiben.

1.16 16. Erweiterungsaufgaben für besonders schnelle Schüler

Die folgenden Aufgaben sind bewusst in drei Schwierigkeitsstufen geordnet. Jede Aufgabe kann unabhängig bearbeitet werden. Die Schülerinnen und Schüler sollten vor dem Programmieren zuerst

festlegen, **welcher Spielwert gespeichert wird, wann die Prüfung stattfinden soll und woran man im Spiel erkennt, dass die Erweiterung funktioniert.**

1.16.1 Stufe A – sicherer Einstieg in die vorhandene Struktur

Aufgabe	Arbeitsauftrag	Abnahmekriterium	Lernziel
A1 – Eigene einfache Mission	Ergänze eine Mission für mindestens 100 Holz, 50 Nahrung oder 20 Bewohner.	Die Mission erscheint im Menü, wird abgeschlossen und zahlt genau einmal eine Belohnung aus.	Dictionaries und Bedingungen
A2 – Gebäudespezialist	Erstelle eine Mission für drei Gebäude einer selbst gewählten Gebäudesorte.	Der Zähler unterscheidet die Gebäudesorte korrekt von allen anderen Gebäuden.	Zähler und Dictionary-Schlüssel
A3 – Missionsrangliste	Zeige oben im Menü zusätzlich „Bronze“, „Silber“ oder „Gold“ an. Bronze gibt es ab 5, Silber ab 10 und Gold ab 15 erledigten Missionen.	Der Rang verändert sich automatisch mit dem Fortschritt.	if/elif und Vergleichsoperatoren
A4 – Fortschrittsbalken	Zeichne unter jedem offenen Missionsziel einen farbigen Balken. Der Balken soll bei 0 % kurz und bei 100 % vollständig sein.	Der Balken wächst sichtbar und wird nie länger als sein Rahmen.	Prozentrechnung und Pygame-Rechtecke
A5 – Missionen filtern	Ergänze Tasten für „Alle“, „Offen“ und „Erledigt“.	Die Anzeige zeigt nur die jeweils ausgewählte Gruppe.	Listen, boolesche Werte und Eingaben
A6 – Belohnungswahl	Lass die Spielperson bei einer Mission zwischen zwei Belohnungen wählen, zum Beispiel +100 Gold oder +50 Forschung.	Die Auswahl wird nur einmal angezeigt und die gewählte Belohnung wird gespeichert.	Menüzustand und Verzweigungen

1.16.2 Stufe B – mehrere Grundideen kombinieren

Aufgabe	Arbeitsauftrag	Abnahmekriterium	Lernziel
B1 – Mission mit UND und ODER	Erweitere die Bedingungen so, dass eine Mission entweder zwei Ziele gleichzeitig oder eines von zwei alternativen Zielen akzeptiert.	Eine Mission kann zum Beispiel „Farm UND 20 Bewohner“ oder „Reaktor ODER Forschung“ abbilden.	Funktionen, <code>all(...)</code> und <code>any(...)</code>
B2 – Missionsketten	Erstelle drei aufeinander aufbauende Missionen: „Baue eine Farm“, danach „Erzeuge Nahrung“, danach „Versorge 20 Bewohner“.	Die nächste Mission wird erst freigeschaltet, wenn die vorherige erledigt ist.	Abhängigkeiten und Zustandsprüfung
B3 – Zufällige Ereigniskarten	Erzeuge alle 20 Wirtschaftsticks ein zufälliges Ereignis, zum Beispiel Meteoritensturm, gute Ernte oder Handelsbesuch.	Die Karte zeigt zwei Entscheidungen mit unterschiedlichen Ressourcenfolgen.	<code>random</code> , Zustände und <code>if</code> -Verzweigungen
B4 – Missionskategorien	Ergänze Kategorien wie „Bauen“, „Forschung“, „Handel“ und „Versorgung“ und zeige die Kategorie im Menü an.	Die Aufgaben können nach Kategorie sortiert oder gefiltert werden.	Datenmodell erweitern und Listen verarbeiten
B5 – Missionsstatistik	Zeige neben den erledigten Missionen die Gesamtzahl der erhaltenen Belohnungen und die längste Missionsserie an.	Die Statistik bleibt nach Speichern und Laden erhalten.	Zähler, Funktionen und JSON

Aufgabe	Arbeitsauftrag	Abnahmekriterium	Lernziel
B6 – Nachrichtenschlange	Mehrere gleichzeitig erledigte Missionen sollen nacheinander angezeigt werden, statt dass die letzte Meldung alle anderen überschreibt.	Jede Erfolgsmeldung ist mindestens kurz sichtbar und wird in der richtigen Reihenfolge abgearbeitet.	Listen als Warteschlange und Zeitsteuerung

1.16.3 Stufe C – anspruchsvolle Abschlussaufgaben

Aufgabe	Arbeitsauftrag	Abnahmekriterium	Lernziel
C1 – Tagesmission	Erzeuge pro Spielstart eine zufällig ausgewählte Tagesmission. Sie darf eine höhere Belohnung geben, wird aber nur einmal pro Spieltag abgeschlossen.	Der Tagesauftrag wechselt kontrolliert und wird nicht bei jedem Frame neu ausgewürfelt.	Zufall, Initialisierung und persistenter Zustand
C2 – Eigene Missionen aus JSON	Lade zusätzliche Missionen aus einer Datei <code>eigene_missionen.py</code> .	Eine neue Mission kann durch Bearbeiten der JSON-Datei hinzugefügt werden, ohne <code>missionen.py</code> zu verändern.	JSON, Fehlerbehandlung und Datenvalidierung
C3 – Spielstand-Versionen	Ergänze eine Versionsnummer für Missionsdaten und schreibe eine kleine Umwandlung für alte Spielstände ohne Missionsfeld.	Ein alter Spielstand kann weiterhin geladen werden und erhält leere Missionsdaten als Standard.	Kompatibilität und defensive Programmierung
C4 – Missioneneditor	Baue einen einfachen Editor, in dem eine Person Titel, Beschreibung, Zielwert und	Der Editor speichert eine gültige neue Mission und das Spiel kann sie laden.	Eingabeverarbeitung, Dictionaries und Validierung

Aufgabe	Arbeitsauftrag	Abnahmekriterium	Lernziel
	Belohnung eingeben kann.		
C5 – Dynamische Belohnungen	Die Belohnung steigt je nach Schwierigkeit oder Fortschritt, zum Beispiel Zielwert * 5 Gold.	Die Belohnung wird korrekt berechnet, bleibt nachvollziehbar und wird nur einmal vergeben.	Formeln, Variablen und Funktionen
C6 – Automatisierte Tests	Schreibe für mindestens drei eigene Missionen Tests für offen, gerade erfüllt und bereits erledigt.	Die Tests laufen ohne Pygame-Fenster und melden Fehler eindeutig.	Testdenken und reproduzierbare Fehlerprüfung
C7 – Missionsarchiv	Erstelle eine zweite Ansicht mit Datum, Abschlussreihenfolge und ausgezahlter Belohnung.	Das Archiv zeigt abgeschlossene Missionen auch dann noch, wenn die aktuelle Liste später erweitert wird.	Listen von Dictionaries und persistente Historie
C8 – Balancing-Bericht	Vergleiche Zielwerte und Belohnungen und schreibe eine kurze Begründung, ob eine Mission zu leicht oder zu schwer ist.	Die Schülerin oder der Schüler verändert mindestens drei Werte begründet und dokumentiert die Wirkung.	Datenanalyse, Modellierung und Reflexion

1.16.4 16.1 Besonders geeignete Abschlussprojekte

Für eine sehr schnelle Person eignen sich besonders **B2 Missionsketten**, **B3 Zufällige Ereigniskarten** oder **C2 Eigene Missionen aus JSON**. Diese Aufgaben wirken im Spiel deutlich sichtbar, bleiben aber mit Grundkenntnissen in Python lösbar. Eine gute Kombination wäre eine Missionskette mit einem zufälligen Ereignis: Das Ereignis verändert Ressourcen, und die Missionskette prüft, wie die Kolonie mit der Situation umgeht.

Für Partnerarbeit kann eine Person das Datenmodell und eine zweite Person das Menü bearbeiten. Beide Teile können anschließend über eine klar definierte Funktion wie `missionen_pruefen(kontext)` verbunden werden. Wichtig ist, dass die Gruppe vorher die Namen der Dictionary-Schlüssel festlegt. So verhindert sie, dass eine Seite `bewohner` und die andere Seite `bevoelkerung` verwendet.

1.16.5 16.2 Bewertungsraster für Erweiterungen

Kriterium	Erfüllt	Teilweise erfüllt	Noch offen
Die Erweiterung startet ohne Absturz.	Funktioniert in mehreren Starts.	Funktioniert nur in einem Ablauf.	Programm bricht ab.
Die Datenstruktur ist verständlich.	Namen und Kommentare sind klar.	Einzelne Stellen sind schwer lesbar.	Werte sind verstreut oder doppelt.
Die Funktion wird im Menü sichtbar.	Fortschritt und Ergebnis sind klar.	Anzeige ist unvollständig.	Es gibt keine sichtbare Rückmeldung.
Der Spielstand bleibt korrekt.	Speichern und Laden wurden getestet.	Nur Speichern oder Laden getestet.	Fortschritt geht verloren.
Der Code ist getestet.	Eigener Testfall vorhanden.	Nur manuelles Testen.	Keine Prüfung dokumentiert.
Die Lösung ist erklärt.	Kurze Begründung und Kommentare vorhanden.	Nur einzelne Kommentare vorhanden.	Keine Erklärung vorhanden.

1.16.6 16.3 Regel für selbstständiges Arbeiten

Vor jeder Erweiterung sollte eine kurze Planung geschrieben werden:

1. Welche Daten brauche ich?
2. Welche Funktion prüft die Daten?
3. Wann wird die Funktion aufgerufen?
4. Was sieht die Spielperson im Menü?
5. Was wird im Spielstand gespeichert?
6. Wie teste ich Erfolg, Misserfolg und Wiederholung?

Diese sechs Fragen verhindern, dass zunächst nur eine Anzeige gebaut wird, die noch nicht mit dem eigentlichen Spielzustand verbunden ist. Erst wenn Daten, Prüfung, Anzeige und Speichern zusammenpassen, ist die Erweiterung vollständig.

1.17 17. Forschungshinweise und Eisen als Entwicklungsweg

1.17.1 17.1 Warum die Fehlermeldung wichtig ist

Beim Versuch, ein Gebäude zu bauen, prüft das Spiel jetzt drei Dinge nacheinander:

1. Ist die notwendige Forschung abgeschlossen?
2. Ist der Bauplatz frei und für das Gebäude geeignet?
3. Sind die Baurohstoffe vorhanden?

Diese Reihenfolge ist wichtig. Ein Gebäude kann gleichzeitig wenig Rohstoffe kosten und trotzdem noch gesperrt sein. Dann wäre die Meldung „Rohstoffe fehlen“ falsch. Das Spiel meldet deshalb beispielsweise:

Eisenmine: Forschung fehlt: Eisenminenbau

Im Baumenü wird derselbe Hinweis neben dem gesperrten Gebäude angezeigt. Der Code dafür befindet sich in `ressourcen.py` in `freischaltung_hinweis()`. Die Funktion `fehlende_baukosten()` wird erst danach verwendet und nennt die konkret fehlenden Mengen.

1.17.2 17.2 Der neue Eisen-Forschungspfad

Eisen entwickelt sich in mehreren Stufen. Die normale Mine ist früh erreichbar, aber ihre Eisenerunde sind zufällig. Wer zuverlässig Eisen erzeugen möchte, muss später mehrere Forschungen abschließen.

Stufe	Was wird gemacht?	Wirkung
1	Minenbau erforschen	Die normale Mine wird freigeschaltet.
2	Eine Mine bauen	Sie produziert Kohle und findet gelegentlich Eisen.
3	Eisenerkundung erforschen	Die Fundchance der normalen Mine steigt deutlich.
4	Stahlverarbeitung erforschen	Das Stahlwerk wird freigeschaltet und die Metallurgie wird vorbereitet.
5	Eisenminenbau erforschen	Teure Spezialforschung; sie benötigt Eisenerkundung und Stahlverarbeitung.
6	Eisenmine bauen	Die Eisenmine produziert zuverlässig Eisen.

Die Werte sind im Spiel wie folgt festgelegt:

Situation	Eisenchance der normalen Mine
Ohne Zusatzforschung	10 % pro Wirtschaftstick
Mit Eisenerkundung	35 % pro Wirtschaftstick
Mit Eisenerkundung und Tiefenbohrung	45 % pro Wirtschaftstick

Eine Chance von 35 % bedeutet nicht, dass genau nach drei Wirtschaftsticks Eisen gefunden wird. Jeder Wirtschaftstick wird neu zufällig geprüft. Mehrere erfolglose Versuche hintereinander sind daher möglich. Genau das macht die normale Mine zu einer frühen, aber unsicheren Lösung.

1.17.3 17.3 Die echte Eisenmine

Die Eisenmine ist absichtlich eine spätere Belohnung für Forschung. Sie kostet:

```
{  
    "gold": 160,  
    "energie": 40,
```

```
"stein": 60,  
}
```

Nach dem Bau produziert sie pro Wirtschaftstick 3 Eisen, verbraucht 4 Energie und benötigt 3 Personal. Im Unterschied zur normalen Mine verwendet sie keine Zufallschance. Sie ist dadurch teurer, aber planbar. Die Zufallschance der normalen Mine ist als eigene Funktion geschrieben:

```
def eisen_fund_chance():  
    chance = 0.10  
  
    if forschung.ist_technologie_erforscht("eisenerkundung"):  
        chance += 0.25  
  
    if forschung.ist_technologie_erforscht("tiefenbohrung"):  
        chance += 0.10  
  
    return min(1.0, chance)
```

Darin stecken mehrere wichtige Python-Ideen. `chance` ist eine Variable, die mit `0.10` startet. Die beiden `if`-Bedingungen prüfen, ob bestimmte Forschungen abgeschlossen sind. Mit `+=` wird ein Bonus addiert. `min(1.0, chance)` verhindert, dass die Chance größer als 100 % wird.

1.17.4 17.4 Mögliche Schüleraufgaben zum Eisen-System

Eine schnelle Schülerin oder ein schneller Schüler kann den Eisenpfad weiterentwickeln. Geeignete Aufgaben sind:

1. Ergänze eine Anzeige im Forschungsmenü, die die aktuelle Eisenchance in Prozent anzeigt.
2. Zeichne beim Hover über die normale Mine den Text „Eisenchance: 35 %“.
3. Erstelle ein eigenes Bild für die Eisenmine und ersetze den vorläufig verwendeten Mine-Platzhalter.
4. Ergänze eine Forschung „Erzveredelung“, die aus 2 Eisen 1 Stahl erzeugt.
5. Führe für `eisen_fund_chance()` Tests mit keiner, einer und zwei Forschungen durch.
6. Ergänze eine Mission „Metallurgische Unabhängigkeit“, die den Bau einer Eisenmine verlangt.
7. Füge eine Warnung hinzu, wenn die Eisenmine wegen fehlender Energie oder fehlendem Personal nicht arbeitet.

Bei jeder dieser Aufgaben sollte zuerst festgelegt werden, ob die Änderung nur die Anzeige, die Spiellogik, den Spielstand oder mehrere dieser Bereiche betrifft.

1.18 18. Zwei Schülergebäude: Park und Solarreaktor

Die beiden Gebäude in diesem Abschnitt wurden von Schülerinnen und Schülern gestaltet. Die Grafiken liegen im Ordner `bilder/` und werden beim Start durch `gebäude_initialisieren()` geladen. Weil die Bilddateien klein sind, werden sie wie alle anderen Gebäudebilder für die Anzeige vergrößert und anschließend aus dem Bild-Cache wiederverwendet.

1.18.1 18.1 Der Park

Ein Park kostet **35 Gold, 20 Holz und 15 Stein**. Er benötigt kein Personal und produziert selbst keine Ressource. Seine besondere Funktion wirkt auf die gesamte Kolonie: Jeder Park senkt den

Nahrungsverbrauch um 5 Prozentpunkte. Der Bonus ist bei 30 Prozent gedeckelt, damit Parks nicht dazu führen, dass Nahrung vollständig kostenlos wird.

Die Funktion ist in `ressourcen.py` bewusst als eigene Funktion geschrieben:

```
def park_nahrungsfaktor(liste_gebaeude):
    anzahl_parks = sum(g.get("typ") == 19 for g in liste_gebaeude)
    return max(0.70, 1.0 - anzahl_parks * 0.05)
```

Die `sum(...)`-Funktion zählt hier, wie viele Dictionaries in der Gebäudeliste den Typ 19 besitzen. Der Ausdruck `g.get("typ") == 19` ergibt entweder `True` oder `False`. Python zählt `True` dabei wie 1 und `False` wie 0. Der Rückgabewert wird später mit dem normalen Nahrungsverbrauch multipliziert.

1.18.2 18.2 Der Solarreaktor

Der Solarreaktor kostet **90 Gold, 10 Energie und 25 Stein**. Er benötigt ein Personal und produziert pro Wirtschaftstick **10 Energie**. Im Gegensatz zum normalen Reaktor verbraucht er keine Kohle. Dadurch eignet er sich besonders für Kolonien, die eine erneuerbare Energiequelle aufbauen wollen.

Der Solarreaktor wird in der allgemeinen Gebäudewirtschaft als Dictionary beschrieben:

```
{
    "baukosten": {"gold": 90, "energie": 10, "stein": 25},
    "produktion": {"energie": 10},
    "verbrauch": {},
    "personalbedarf": 1,
}
```

Das Spiel kann dieses Dictionary in der normalen Produktionsschleife verwenden. Dadurch muss für jedes neue Gebäude keine eigene große Sonderfunktion geschrieben werden. Die besondere Parkwirkung benötigt dagegen eine eigene Funktion, weil sie global auf die Nahrung der gesamten Kolonie wirkt.

1.18.3 18.3 Missionen und Achievements

Für die neuen Gebäude gibt es zusätzliche Ziele:

Ziel	Bedingung	Belohnung
Grüne Oase	Baue einen Park.	50 Nahrung und 25 Gold
Sonnenkraft	Baue einen Solarreaktor.	75 Energie und 25 Forschung
Grüne Oase als Achievement	Baue einen Park.	20 Achievement-Punkte
Solarpionier als Achievement	Baue einen Solarreaktor.	30 Achievement-Punkte

Die Missionen werden in `missionen.py` ergänzt. Die Achievements stehen in `achievements.py`. Beide Systeme nutzen die Gebäude-Typnummern 19 und 20. Die alten Typnummern wurden nicht verschoben, damit ältere Spielstände ihre Gebäude weiterhin richtig laden.

1.18.4 18.4 Mögliche Erweiterungsaufgaben

1. Ergänze eine Forschung „Stadtparks“, die den maximalen Parkbonus von 30 auf 50 Prozent erhöht.
2. Füge einen zweiten Parktyp hinzu, der zusätzlich ein wenig Bevölkerung erzeugt.

3. Erstelle eine Nacht-Mechanik, bei der Solarreaktoren nachts weniger Energie produzieren.
4. Ergänze eine Batterie, die überschüssige Energie des Solarreaktors speichert.
5. Zeige im HUD an, wie viele Parks gebaut sind und wie hoch der aktuelle Nahrungsbonus ist.
6. Erstelle ein eigenes Bild für die Energieanzeige oder eine kleine Animation für den Solarreaktor.
7. Schreibe einen Test, der überprüft, dass zehn Parks nicht mehr als 30 Prozent Nahrungsbonus geben.

1.19 19. Neue Ressource: Zufriedenheit

Die Kolonie besitzt jetzt neben Gold, Energie, Holz, Stein, Nahrung und Forschung eine weitere Ressource: **Zufriedenheit**. Sie beschreibt, wie wohl sich die Bewohnerinnen und Bewohner in der Kolonie fühlen. Zufriedenheit startet bei **0**. Anders als die meisten Rohstoffe darf sie auch negativ werden. Das Spiel begrenzt sie jedoch immer auf den Bereich:

```
-50 <= Zufriedenheit <= +50
```

Ein negativer Wert bedeutet, dass die Kolonie belastet ist. Ein positiver Wert bedeutet, dass die Kolonie besonders angenehm ist. Die Anzeige im HUD verwendet ein Pluszeichen für positive Werte, zum Beispiel +18, und ein Minuszeichen für negative Werte, zum Beispiel -7.

1.19.1 19.1 Gebäude beeinflussen Zufriedenheit

Jedes Gebäude besitzt in `ressourcen.py` einen Grundbeitrag:

```
GEBAEUDE_ZUFRIEDENHEIT = {
    1: -3,    # Reaktor
    6: 3,     # Wohnhaus
    12: -2,   # Stahlwerk
    19: 6,    # Park
    20: 4,    # Solarreaktor
}
```

Die Nummern sind die Gebäude-Typnummern. Positive Zahlen verbessern die Zufriedenheit, negative Zahlen belasten sie. Der Park hat mit +6 einen besonders starken positiven Beitrag. Ein Reaktor hat mit -3 einen negativen Beitrag, weil er Lärm verursacht und Kohle benötigt. Ein Solarreaktor verbessert die Zufriedenheit, weil er saubere Energie erzeugt.

Die vollständige Verteilung lautet:

Gebäudeart	Beitrag pro Wirtschaftstick vor der Abschwächung
Park	+6
Wohnblock	+4
Solarreaktor	+4
Wohnhaus	+3
Farm, Universität und Gewächshaus	+2

Gebäudeart	Beitrag pro Wirtschaftstick vor der Abschwächung
Marktplatz, Straße, Lagerhaus und Handelsposten	+1
Basis	0
Holzfäller, Steinmetz, Roboterfabrik und Eisenmine	-1 bis -2
Reaktor, Stahlwerk und Fusionsreaktor	-2 bis -4
Koloniezentrum	+5

1.19.2 19.2 Warum verändert sich der Wert langsam?

Der Grundbeitrag wird nicht sofort vollständig auf die Ressource addiert. Stattdessen wird pro Wirtschaftstick nur ein Viertel verwendet. Ein Park mit Grundbeitrag +6 verändert den aktuellen Wert also um +1.5 pro Tick. Dadurch springt die Anzeige nicht plötzlich von 0 auf 50, sondern entwickelt sich sichtbar und planbar.

```
veraenderung = grundbeitrag * 0.25
ressourcen_dict["zufriedenheit"] += veraenderung
```

Wenn kein Gebäude einen Beitrag liefert, nähert sich der Wert langsam wieder 0 an. So bleibt ein alter Extremwert nicht für immer bestehen. Nach jeder Änderung wird erneut geprüft, ob der Wert zwischen -50 und +50 liegt.

1.19.3 19.3 Zwei Grenzen, die nicht verwechselt werden dürfen

Die Zufriedenheitsgrenzen sind keine normalen Speichergrenzen. Gold oder Eisen können nicht negativ werden und werden gegen 0 begrenzt. Zufriedenheit darf dagegen bis -50 sinken und bis +50 steigen.

```
ZUFRIEDENHEIT_MIN = -50.0
ZUFRIEDENHEIT_MAX = 50.0
```

Die Funktion `zufriedenheit_begrenzen()` sorgt dafür, dass auch fehlerhafte Werte aus einem Spielstand korrigiert werden. Dadurch kann eine beschädigte Eingabe nicht zu einer Zufriedenheit von +200 führen.

1.19.4 19.4 Park als mehrfache Verbesserung

Der Park besitzt zwei voneinander unabhängige Funktionen. Erstens erhöht er Zufriedenheit. Zweitens senkt er den globalen Nahrungsverbrauch. Jeder Park reduziert den Verbrauch um 5 Prozentpunkte; bei 30 Prozent ist der Bonus gedeckelt.

Das bedeutet, dass ein Park auch dann sinnvoll bleibt, wenn die Kolonie bereits ausreichend Nahrung besitzt. Er verbessert weiterhin die Zufriedenheit. Gleichzeitig muss die Spielperson abwägen, ob ein Park mehr Nutzen bringt als ein weiteres Industriegebäude.

1.19.5 19.5 Mission und Achievement

Ab einer Zufriedenheit von +30 werden zwei Ziele relevant:

System	Ziel	Belohnung
Mission	Zufriedene Kolonie mit mindestens +30	100 Gold und 50 Nahrung
Achievement	Zufriedene Kolonie mit mindestens +30	45 Punkte

Die Mission verwendet den Schlüssel `zufriedenheit` im Missionskontext. In `main.py` wird der aktuelle Wert mit folgender Zeile bereitgestellt:

```
zaehler["zufriedenheit"] = ressourcen_dict.get("zufriedenheit", 0)
```

1.19.6 19.6 Mögliche Schülererweiterungen

1. Zeige im HUD zusätzlich an, wie viele Gebäude gerade positiv und wie viele negativ wirken.
2. Erstelle eine Warnung bei Zufriedenheit unter -25 und eine Erfolgsmeldung ab +25.
3. Füge eine neue Forschung „Sozialplanung“ hinzu, die negative Gebäude-Beiträge um 25 Prozent abschwächt.
4. Ergänze ein Gebäude „Freizeitzentrum“, das viel Zufriedenheit erzeugt, aber Nahrung und Energie verbraucht.
5. Verändere die Siegbedingungen so, dass in einer Regelvariante zusätzlich mindestens +20 Zufriedenheit benötigt wird.
6. Füge ein Achievement für -40 Zufriedenheit hinzu: „Das Volk ist unzufrieden“.
7. Schreibe Tests für den Minimalwert, den Maximalwert und die langsame Rückkehr zu 0.
8. Erstelle ein Diagramm, das den Verlauf der Zufriedenheit über 30 Wirtschaftsticks darstellt.